

devreport

Turn a messy project folder into a polished PDF report or slide deck

Celibistrial

October 2026

devreport takes whatever a student has lying around from a project (dev logs, notes, a CSV, screenshots, a .zip, or a GitHub link), analyses it with plain code, asks up to three optional questions it can't answer from the files, and then has Claude Code write and compile a LaTeX report or Beamer deck, live in the browser.

1 *Problem and who it's for*

devreport is for student builders who have to write up a hackathon, coursework or side project and would rather keep building. The material for a write-up usually exists already, scattered across git history, logs, notes and screenshots. What takes time is pulling it together into a document that reads well and doesn't overstate anything.

2 *What I built*

devreport is a local web app. The user drops files into the page or pastes a public GitHub link, picks a report or slides, picks a theme, and optionally answers what problem the project solves, what they learned and what's next. A later version added a fourth optional question about the result they are proudest of, so the Results section has a real source. The page then shows a live feed of what Claude is doing and displays `main.pdf` inline when it is done, with the `.tex` available to download.

There are two themes, Paper (serif, academic) and Midnight (dark, neon, terminal). Each is a single `.sty` file that works for both reports and slides, charts included. For slides, a user can also upload their own PowerPoint template: `pptx.js` pulls out its colour palette, heading and body fonts, logo, title and body box positions and a preview thumbnail into `template/template.json`, and Claude writes a matching Beamer theme from it, compiles a test slide, compares it with the thumbnail and adjusts.

3 *How it works*

The pipeline has four stages:

1. An upload, zip or GitHub link goes into a new job folder, `jobs/<id>/input/`.
2. `collect.js` parses the git history, logs, tables, notes and images into `data/*.csv` and `facts.json`.
3. The optional answers are saved as `answers.md`.
4. `claude -p` runs in the job folder: it writes `main.tex`, compiles it with `tectonic`, fixes errors, humanizes the prose and recompiles.

`server.js` is a plain Node HTTP server built only on the standard library, so there is no `npm install`. It parses multipart uploads itself, unpacks zips, clones GitHub repos as a single branch, runs the collector and then spawns `claude -p` with stream-json output. It reads that JSON stream line by line and turns each tool call into a human-readable step (“Reading `facts.json`”, “Compiling with `tectonic`”, “Compile error, fixing”), which it pushes to the browser over server-sent events. The run has a 10-minute timeout, and the server takes one job at a time.

`collect.js` does the analysis deterministically. From the git log (`--numstat`) it builds per-day commit counts, picks up to 8 milestones (the first commit, tagged commits, then the biggest feature commits spread across the project’s time span), and leaves out lockfiles and vendored or generated paths such as `node_modules/`, `dist/` and `.min.js` files. It also finds the likely entry points of the code, groups recurring log errors by normalizing numbers and IDs, and detects column types in the user’s own CSV or JSON. Because `pgfplots` ignores CSV quoting, it strips commas and quotes from cells when it writes CSVs. `test.js` holds `node:test` tests for the collector and the template extractor, and `index.html` is the whole UI in one file.

4 *Key decisions and trade-offs*

- Charts can’t lie. Every chart is `pgfplots` reading a CSV that `collect.js` computed, such as `data/commits_per_day.csv`. Claude never types a number into a chart, and stat numbers come from `facts.json`.
- My code does the analysis and Claude writes the prose. Commit counts and lines of code are not treated as evidence of anything: they only appear as a timeline or an appendix sentence.
- The agent is locked down. Claude Code runs headless with only the Read, Write and Edit tools (scoped to `./**`), `tectonic` as its one shell command, and `Skill`, so it has no general shell and can only write inside the job folder. Uploaded content is treated as data, never as instructions.
- Intake is defensive. Zips are size-checked before extracting to stop zip bombs, entries that escape the job folder are rejected, and symlinks are deleted. GitHub URLs are validated and cloned without a shell, with a timeout.

- There is deliberately no public hosted version. Each run uses the user's own Claude Code login and takes about 1–3 minutes; a hosted version would run strangers' jobs on one person's subscription.

5 *Timeline*

All milestones below fall on 2 October 2026.

- Added the project plan and handoff notes.
- Added the server: multipart upload, zip and GitHub intake, the collector and Claude run, and SSE progress.
- Added the UI: drop zone, repo link, report or slides choice, themes, questions, live feed and PDF view.
- Added the devreport skill: pipeline rules, tested pgfplots chart snippets and the headless prompt.
- Added the Paper and Midnight themes for reports and slides.
- Collector fixes: convert 16-bit PNGs, strip commas from CSV labels, add day counts.
- Accepted a .pptx template for slides and added a template theme card.
- Added the optional “proudest result” question so Results has a real source.

Repo facts

The repository has 27 commits by Celibistrial, all made on 2026-10-02 (1 active day). Across them, 2102 lines were added and 105 removed, excluding vendored and generated files.